Apertura: viernes, 21 de febrero de 2025, 11:47

📦 Tarea: Sistema de Gestión de Pedidos para Múltiples Almacenes

📖 Descripción del Problema:

- Desarrollar una aplicación cliente-servidor en Java para gestionar pedidos realizados desde diferentes almacenes.
- Los clientes representan almacenes que pueden realizar pedidos de diferentes productos.
- El servidor procesa los pedidos, comprueba el stock y responde si el pedido se puede atender o si hay falta de stock.
- La comunicación es bidireccional, y el servidor maneja múltiples almacenes simultáneamente usando un manejador de hilos.
- El stock y los productos disponibles se configuran mediante un archivo de configuración.

⚖️ Requisitos:

Servidor Socket:

- Escuchar en un puerto especificado y aceptar conexiones de múltiples almacenes.
- Recibir los pedidos enviados por los almacenes (producto y cantidad).
- Comprobar el stock y responder si el pedido se acepta o se rechaza por falta de stock.
- Guardar un registro de cada pedido procesado en un archivo de log.
- Actualizar dinámicamente el stock si se modifica el archivo de configuración.

Clientes Socket:

- Cada almacén puede realizar pedidos aleatorios cada 5 o 10 segundos.
- Los pedidos incluyen el nombre del producto y una cantidad aleatoria.
- Recibir la confirmación o rechazo del pedido y mostrarlo en la consola.

💾 Archivos Utilizados:

Archivo de Configuración (config_stock.properties):

```
producto1=100
producto2=50
producto3=200
producto4=150
producto5=300
```

Archivo de Log (pedidos.log):

```
[2025-02-21 11:30:45] PEDIDO - Almacén: 192.168.1.10 - Producto: producto1 - Cantidad: 20 - ACEPTADO
[2025-02-21 11:35:20] PEDIDO - Almacén: 192.168.1.11 - Producto: producto2 - Cantidad: 60 - RECHAZADO (Stock
insuficiente)
```

🌐 Comunicación Bidireccional:

- El cliente envía el nombre del producto y la cantidad al servidor.
- El servidor procesa el pedido y responde indicando si ha sido aceptado o rechazado.
- La conexión permanece abierta para realizar pedidos periódicamente.

💡 Ejemplo de Ejecución:

Servidor:

```
Servidor de pedidos iniciado en el puerto 12345
Almacén conectado: /192.168.1.10
Almacén conectado: /192.168.1.11
Pedido procesado: producto1, cantidad: 20 - ACEPTADO
Pedido procesado: producto2, cantidad: 60 - RECHAZADO (Stock insuficiente)
```

Cliente Almacén 1:

```
Conectado al servidor de pedidos.
Ingrese su pedido: producto1,20
Respuesta del servidor: Pedido aceptado: producto1 - Cantidad: 20
```

?

Cliente Almacén 2:

```
Conectado al servidor de pedidos.
Ingrese su pedido: producto2,60
Respuesta del servidor: Pedido rechazado: Stock insuficiente.
```

🛠️ Indicaciones de entrega

- Crear un fichero comprimido que incluya:
  - El proyecto Java completo.
  - Un documento explicativo con:
    - Descripción de la solución implementada.
    - Capturas de pantalla del programa en ejecución.
    - Resultados obtenidos.
- **Nomenclatura del archivo:** `PSPXX_TareaYY_apellido1_apellido2_nombre`.
- Ejemplo: `PSP02_Tarea01_Perez_Lopez_Juan.zip`.

✨ Consejos y Recomendaciones

- Usar estructuras como `Queue` para gestionar clientes.
- Manejar concurrencia con hilos y sincronización.
- Implementar un sistema de logs para facilitar la depuración.

📊 Criterios de Calificación: Total 10 puntos

- **(2 puntos):** Implementación correcta del servidor para aceptar peticiones de clientes.
- **(1 punto):** Correcto manejo del archivo de configuración.
- **(2 puntos):** Comunicación bidireccional entre servidor y clientes.
- **(1 punto):** Registro adecuado de pedidos en el archivo log.
- **(1 punto):** Generación correcta de pedidos simulados en los clientes.
- **(1 punto):** Manejo correcto de errores y desconexiones inesperadas de los clientes.
- **(1 punto):** Actualización dinámica del stock.
- **(1 punto):** Buenas prácticas en el código, con comentarios y estructura clara.

🔧 Resultados de aprendizaje y criterios de evaluación

Programa mecanismos de comunicación en red empleando sockets y analizando el escenario de ejecución:

- **a)** Se han identificado escenarios que precisan establecer comunicación en red.
- **b)** Se han identificado los roles de cliente y servidor.
- **c)** Se han reconocido librerías y mecanismos del lenguaje para la comunicación en red.
- **d)** Se ha analizado el concepto de socket, sus tipos y características.
- **e)** Se han utilizado sockets para programar aplicaciones cliente-servidor.
- **f)** Se ha desarrollado una aplicación servidor y verificado su funcionamiento.
- **g)** Se han utilizado hilos para gestionar la concurrencia y la comunicación en red.

Agregar entrega

Estado de la entrega

Número del intento	Este es el intento 1 (2 intentos permitidos).
Estado de la entrega	Todavía no se han realizado envíos
Estado de la calificación	Sin calificar
Última modificación	-